ARDUINO

UNO Q

AI For Edge Devices



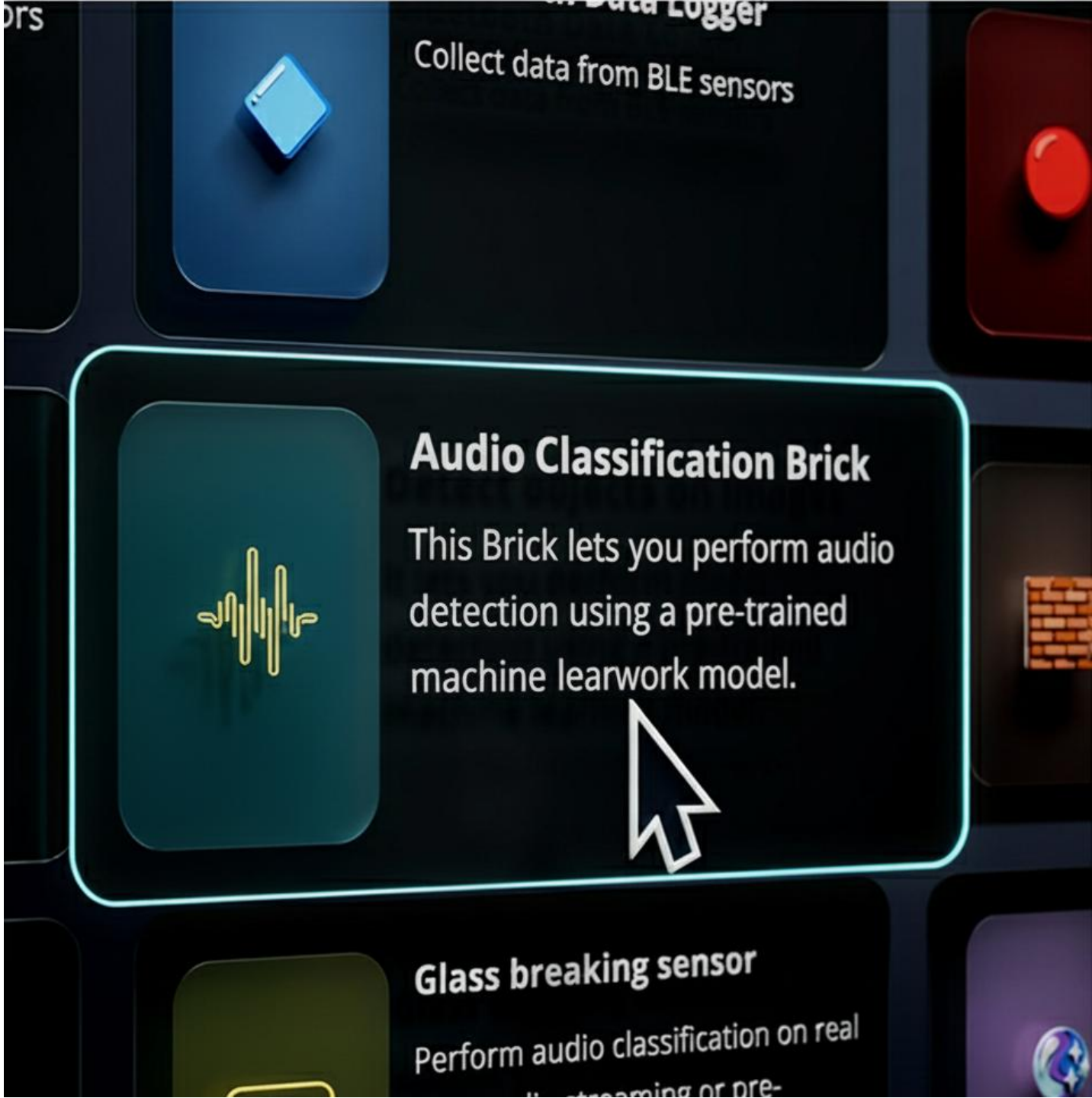
EDGE IMPULSE

Arduino Bricks

UNO Q

Audio Classification






Bricks
ARDUINO
UNO Q
Keyword Spotting
28:07
Arduino UNO Q (Ep.38) วิธีใช้ “Keyword Spotting Brick” บ...
Bricks
ARDUINO
UNO Q
Keyword Spotting
11:56
Arduino UNO Q (Ep.37) วิธีใช้ “Keyword Spotting Brick” บ...
AI For Edge Devices
ARDUINO
UNO Q
EDGE IMPULSE
40:33
Arduino UNO Q (Ep.36) การเทรนโมเดล AI ให้ใช้กับงาน...

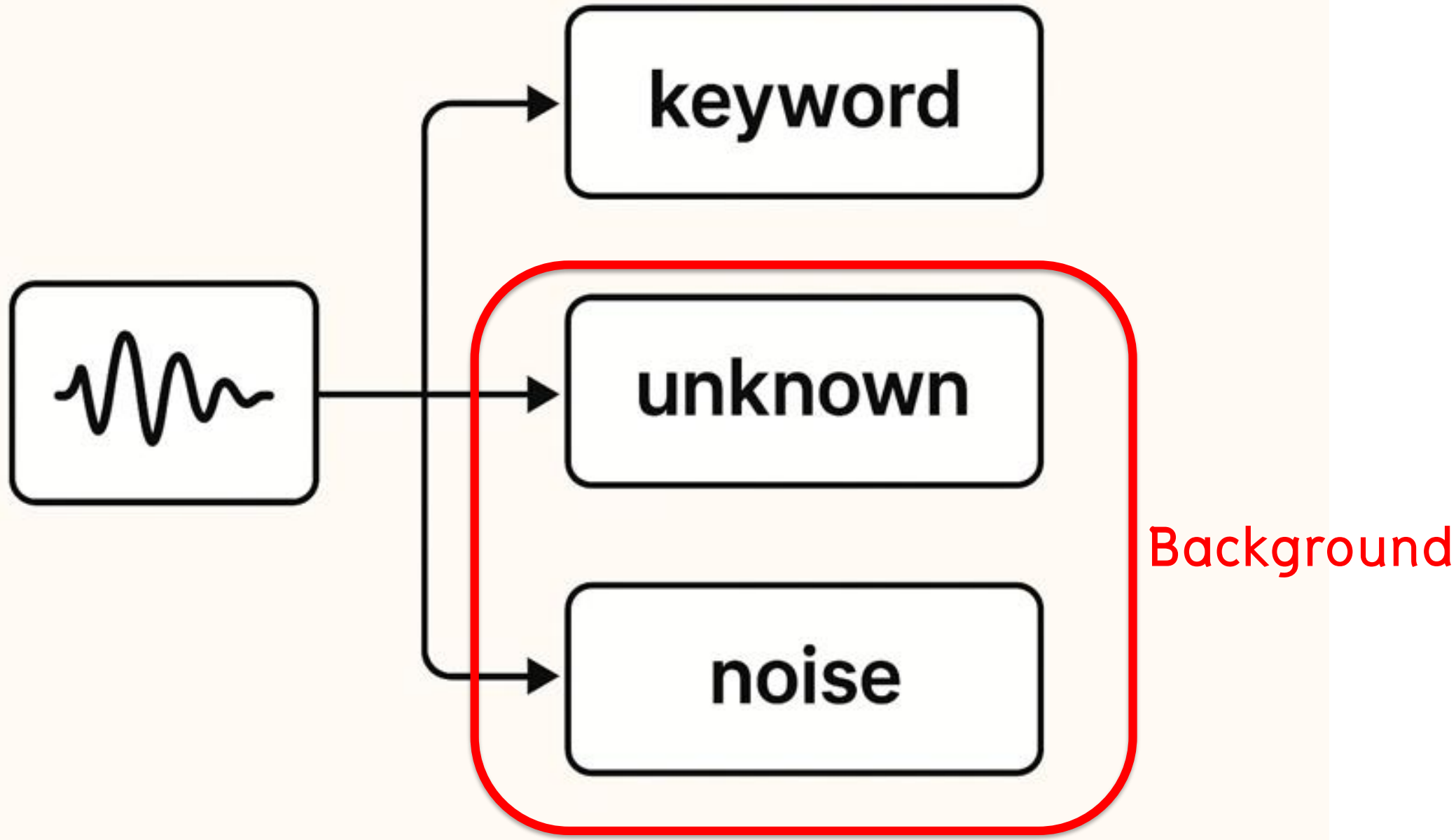

Bricks
ARDUINO
UNO Q
ตรวจจับวัตถุ
Object Detection
16:37
Arduino UNO Q (Ep.20) วิธีใช้ “Object Detection Brick” บน Arduino App Lab ตอนที่...


AI For Edge Devices
ARDUINO
UNO Q
EDGE IMPULSE
15:44
Arduino UNO Q (Ep.33) กระบวนการ Deploy ไฟล์โมเดล (.eim) สู่บอร์ด Arduino UNO Q


AI For Edge Devices
ARDUINO
UNO Q
EDGE IMPULSE
4:46
Arduino UNO Q (Ep.35) ถ่ายซ่อมคลิป Ep.33
การดู 52 ครั้ง • 1 วันที่ผ่านมา

ตารางประเภทคลาสใน Audio Classification

ประเภทคลาส	ความหมาย	ตัวอย่าง
Keyword Class	คลาสคำสั่งที่ต้องการให้โมเดลรู้จำเป็นพิเศษ	“เปิด”, “ปิด”, “ซ้าย”, “ขวา”, “Hey Arduino”
Background/Noise Class/Unknown class	เสียงที่ไม่ใช่เสียงพูด ใช้ฝึกให้โมเดลกันเสียงรบกวนเสียงเงียบ หรือพื้นหลังที่แทบไม่มีเสียง	เสียงลม, เครื่องปรับอากาศ, เสียงรถ, เสียงจานช้อน, ช่วงไม่มีอะไรพูดเลย หรือเกือบเงียบสนิท


keyword
unknown
noise
Background

Keyword คืออะไร?

Keyword คือ คำหรือเสียงเฉพาะที่คุณต้องการให้โมเดลตรวจจับและตอบสนอง

- เป็นเป้าหมายหลัก ของโมเดล
- โมเดลจะเรียนรู้ลักษณะเสียงของ keyword แต่ละคำเพื่อจำแนกจากเสียงอื่น ๆ

จุดสำคัญของ Keyword ใน Audio Classification

1. **เป็นเป้าหมายหลัก**: โมเดลจะเรียนรู้เฉพาะเสียง keyword เพื่อให้ตอบสนองต่อมัน
2. **ต้องแยกจากเสียงอื่น ๆ**: เสียงพูดอื่น ๆ จะถูกจัดเป็น **background**
3. **สามารถมีหลายคำ**: โมเดลสามารถเรียนรู้ keyword หลายคำพร้อมกัน
4. **สำคัญต่อ callback/action**: เมื่อโมเดลตรวจพบ keyword ⟶ trigger function หรือ action ตามที่ตั้งไว้

สรุปสั้น ๆ

Keyword = คำสั่งหรือเสียงเป้าหมายที่เราต้องการให้โมเดลรู้จำและตอบสนอง

Background / Unknown / Noise = เสียงอื่น ๆ ที่ไม่ใช่ keyword เพื่อป้องกัน false positive

งาน	Keyword / คลาสหลัก	Background/Unknown/Noise		จำนวนตัวอย่างแนะนำ / คลาส
ระบบเตือนภัยบ้าน (Home Alarm)	Glass_Breaking, Alarm	คำพูดทั่วไป, เสียงสัตว์	เสียงเครื่องใช้ไฟฟ้า, ลม, น้ำ	Keyword: 150–200 / Background: 100–150
Smart Home / ควบคุมอุปกรณ์ด้วยเสียง	เปิดไฟ, ปิดไฟ, เปิดพัดลม, ปิดพัดลม	คำพูดทั่วไป, ประโยคอื่น	เสียงทีวี, เสียงเครื่องใช้ไฟฟ้า	Keyword : 150 / Background: 100
หุ่นยนต์ควบคุมด้วยเสียง	ซ้าย, ขวา, เดินหน้า, ถอยหลัง	คำพูดอื่น, คำสั่งที่ไม่ได้ใช้	เสียงรบกวนจากเครื่องจักร, พื้นหลัง	Keyword: 120–150 / Background : 100
ตรวจสอบเครื่องจักร / Factory Monitoring	Motor_Fault, Overheat	เสียงเครื่องจักรอื่น, เสียงคนพูด	เสียงพื้นหลังโรงงาน, ลม, พัดลม	Keyword: 100 / Background: 100
ผู้พิการทางการได้ยิน (Assistive Device)	Doorbell, Baby_Crying	เสียงพูด, เสียงสัตว์เลี้ยง	เสียง TV, เสียงรบกวนบ้าน	Keyword: 120 / Background: 100
สัตวแพทย์ / Monitoring สัตว์	Dog_Bark, Cat_Meow, Bird_Chirp	เสียงสัตว์อื่น, เสียงคนพูด	เสียงพื้นหลัง, น้ำไหล, ลม	Keyword: 150 / Background: 100

Keyword ความยาวตัวอย่าง: 5 วินาที sample rate: 16 kHz (mono)

```python
from arduino.app_bricks.audio_classification import AudioClassification
from arduino.app_utils import App

classifier = AudioClassification()

def print_kws_model_info(classifier):
    info = classifier.get_model_info()
    if info is None:
        print("ยังไม่สามารถดึงข้อมูลโมเดลได้")
        return

    # ดู attribute ทั้งหมดของ object
    print("=== ดู attribute ทั้งหมดของ object ===")
    for attr in dir(info):
        # กรอง method และ private attribute
        if not attr.startswith("_") and not callable(getattr(info, attr)):
            print(f"{attr}: {getattr(info, attr)}")


print_kws_model_info(classifier)
```

คำสั่ง get_model_info() มักจะถูกใช้ในคลาส Keyword Spotting (หรือคลาสอื่น ๆ ใน Arduino App Lab ที่ใช้ไฟล์ .eim) เพื่อ ดึงข้อมูล metadata ของโมเดล AI ที่คุณโหลดเข้ามา เช่น ชื่อโมเดล, เวอร์ชัน, อินพุต–เอาต์พุต, รายละเอียดคลาสต่าง ๆ เป็นต้น

คำสั่ง dir() เป็นคำสั่งพื้นฐานใน Python ที่ใช้เพื่อ ดูรายชื่อ attribute และ method ทั้งหมดที่ออบเจ็กต์นั้นมีอยู่

คำสั่ง getattr() เป็นฟังก์ชันพื้นฐานของ Python ที่ใช้เพื่อ ดึงค่า attribute ของออบเจ็กต์ โดยใช้ชื่อเป็นสตริง เหมาะมากเมื่อคุณต้องการเข้าถึง attribute แบบไม่รู้ชื่อมาก่อน (เช่นในลูป)

คำสั่ง callable() ใน Python ใช้เพื่อตรวจสอบว่า ค่าวัตถุ (object) ที่กำลังตรวจอยู่นั้น “เรียกใช้งานได้เหมือนฟังก์ชัน” หรือไม่หรือพูดง่ายๆ คือ “เป็นฟังก์ชันหรือเป็นวัตถุที่เรียก (call) ได้หรือเปล่า

คำสั่ง startswith() เป็นเมธอดของ string ใน Python ที่ใช้เพื่อ ตรวจสอบว่าข้อความ (string) เริ่มต้นด้วยตัวอักษรหรือคำที่กำหนดหรือไม่

```
=== ดู attribute ทั้งหมดของ object ===
axis_count: 1
frequency: 16000
image_input_height: 0
image_input_width: 0
input_features_count: 16000
interval_ms: 0.0625
label_count: 2
labels: ['Background', 'Glass_Breaking']
model_type: classification
name: Glass breaking
project_id: 749446
thresholds: []
2025-11-27 04:26:08.263 WARNING - [MainThread] Microphone:  [__del__] Microphone __del__: cleaning up resources.
2025-11-27 04:26:08.263 WARNING - [MainThread] Microphone:  Microphone is not recording, nothing to stop.
exited with code 0
```

หมาเห่า ⟶ ช่วงความถี่ ~500 Hz – 8 kHz

กระจกแตก ⟶ ช่วงความถี่สูง ~3 kHz – 15 kHz

แผ่นดินไหว (สั่นสะเทือน) ⟶ ส่วนใหญ่ความถี่ต่ำ ~0–50 Hz

AudioClassification(confidence = 0.8)

on_detect() start() stop()

classify_from_file(พาทของไฟล์.wav, confidence = 0.8)

Test and tune your impulse with realistic data

Run test | Options

Select and save a config

Your impulse can be configured with a post-processing algorithm that will minimize either false activations or false rejections. The chart shows a range of suggested configs. If you save one, it will be used when your impulse is deployed.



โมเดลทำนายผิดพลาดว่า “มีเสียง” ทั้งที่ไม่มีเสียง

เฉลี่ยโมเดลทำนายผิดพลาดว่า “มีเสียง” 23% ของเวลา

โมเดลไม่รู้จักเสียงที่ควรตรวจจับ (พลาด)

Selected config

Mean FAR ⓘ: **23%**
Mean FRR ⓘ: **68.4%**

โมเดลพลาดไม่รู้จักเสียงเป้าหมาย 68.4% ของเวลา (ค่อนข้างสูง → โมเดลยังไม่แม่น)

Averaging window duration ⓘ: **97 ms**

ระบบจะเอาผลจากหน้าต่างสั้น 97 ms มาคำนวณค่าเฉลี่ยก่อนตัดสินใจ

Detection threshold ⓘ: **0.50**

ค่า confidence ที่โมเดลต้องถึงเพื่อยืนยันว่ามีเสียง

Suppression period ⓘ: **632 ms**

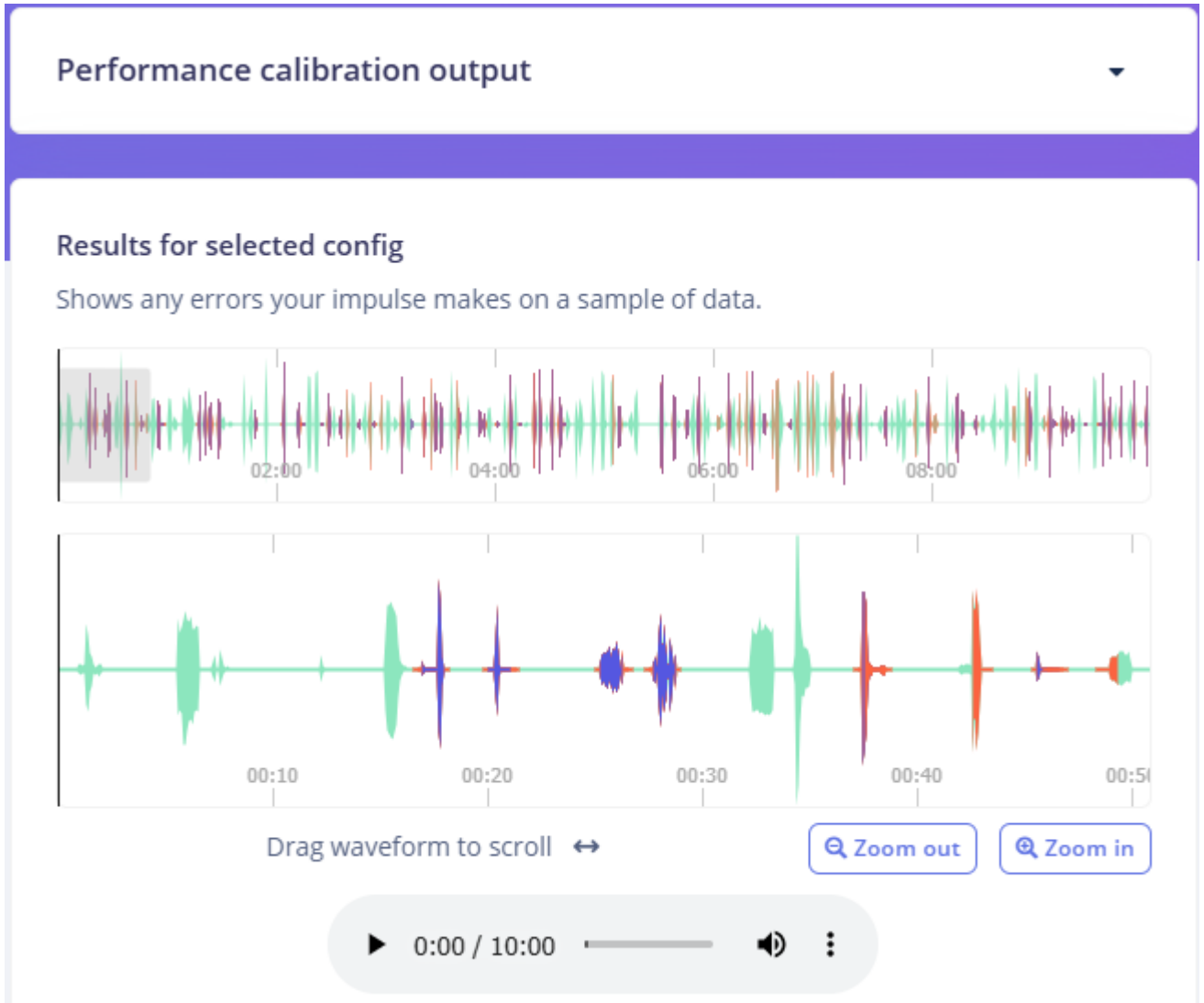
หลังจากตรวจเจอครั้งหนึ่ง โมเดลจะไม่ยืนยันซ้ำภายใน 632 ms เพื่อป้องกันการ activate ซ้ำ

Performance overview

Shows the performance statistics for each label.

LABEL	FAR	FRR	TRUE POSITIVES	FALSE POSITIVES	TRUE NEGATIVES	FALSE NEGATIVES
dog_barking	23%	68.4%	31	26	87	67

สถิติ	ค่า	คำอธิบายและความหมาย
LABEL (ป้ายกำกับ)	dog_barking	ชื่อของเหตุการณ์, วัตถุ, หรือคลาสที่แบบจำลองกำลังพยายามระบุ
FAR (False Accept Rate)	23%	อัตราการยอมรับที่ผิดพลาด (หรือเทียบเท่ากับ False Positive Rate – FPR) คือ **อัตราที่ระบบระบุว่ามีการเห่าของสุนัข ทั้งที่จริง ๆ แล้วไม่มี**
FRR (False Reject Rate)	68.4%	อัตราการปฏิเสธที่ผิดพลาด (หรือเทียบเท่ากับ False Negative Rate – FNR) คือ **อัตราที่ระบบพลาดที่จะระบุว่ามีการเห่าของสุนัข ทั้งที่จริง ๆ แล้วมีการเห่า** (ยิ่งค่านี้สูงยิ่งไม่ดี)
TRUE POSITIVES (TP)	31	จำนวนครั้งที่แบบจำลอง **ระบุถูกต้อง** ว่ามีการเห่าของสุนัข (มีการเห่าจริง และระบบก็ระบุว่ามี)
FALSE POSITIVES (FP)	26	จำนวนครั้งที่แบบจำลอง **ระบุผิดพลาด** ว่ามีการเห่าของสุนัข ทั้งที่จริง ๆ แล้ว **ไม่มี** การเห่า
TRUE NEGATIVES (TN)	87	จำนวนครั้งที่แบบจำลอง **ระบุถูกต้อง** ว่าไม่มีการเห่าของสุนัข (ไม่มีการเห่าจริง และระบบก็ระบุว่าไม่มี)
FALSE NEGATIVES (FN)	67	จำนวนครั้งที่แบบจำลอง **ระบุผิดพลาด** ว่าไม่มีการเห่าของสุนัข ทั้งที่จริง ๆ แล้ว **มีการเห่า** (ระบบพลาด)



นี่คือกราฟรูปคลื่นเสียง (Audio Waveform) ที่แสดงข้อมูลทั้งหมด 10 นาที (ตามที่ระบุที่ด้านล่างของในรูปคือ 0:00 / 10:00) และแสดงตำแหน่งที่ระบบมีการตรวจจับหรือทำนายผล โดยใช้สีต่าง ๆ แทนผลลัพธ์

แถบสีต่าง ๆ ที่อยู่บนกราฟ: แทนการทำนายของระบบ หรือเหตุการณ์ที่ถูกติดป้ายกำกับ (Labels)การวิเคราะห์ข้อผิดพลาด

แท่งที่แสดงเหนือหรือใต้ข้อมูลเสียง มักจะใช้เพื่อระบุ ข้อผิดพลาด (Errors) ในการทำนายของระบบเมื่อเทียบกับความจริง (Ground Truth) ในชุดข้อมูล ตัวอย่างเช่น สีอาจแทน True Positives, False Positives, และ False Negatives ตลอดช่วงเวลา 10 นาที เพื่อให้เห็นภาพรวมว่าระบบทำงานได้ดีในส่วนใด และทำพลาดในส่วนใด

ERROR	TYPE	LABEL	START TIME	
False positive	Incorrect match	dog_barking → Background (sample)	16.5s	Play
False positive	Incorrect match	dog_barking → Background (sample)	16.75s	Play
False negative		dog_barking	16.933s	Play
False positive	Incorrect match	dog_barking → Background (sample)	17s	Play
False positive	Incorrect match	dog_barking → Background (sample)	17.25s	Play
False positive	Incorrect match	dog_barking → Background	19.75s	Play

ตารางแสดง ข้อผิดพลาดรายเหตุการณ์ (Error List) ที่เกิดขึ้นในระหว่างการประเมินประสิทธิภาพของแบบจำลอง (Model) หรือ Impulse

ตารางนี้จะช่วยให้คุณเจาะจงและเข้าใจว่าเหตุการณ์ใดบ้างที่แบบจำลองทำผิดพลาด โดยเฉพาะในช่วงเวลาสั้น ๆ ของข้อมูลเสียง

รายละเอียดของข้อผิดพลาดที่แสดงตารางแสดงข้อมูลสำคัญ 4 คอลัมน์สำหรับแต่ละเหตุการณ์ที่ผิดพลาด:

คอลัมน์คำอธิบาย **ERROR** ประเภทของข้อผิดพลาด ที่เกิดขึ้นในเหตุการณ์นั้น

TYPE ลักษณะของการจับคู่ ระหว่างการทำนายกับความจริง

LABEL ป้ายกำกับที่ถูกทำนาย โดยมีลูกศรแสดงการเปลี่ยนแปลงหรือการพลาด

START TIME เวลาเริ่มต้น ของเหตุการณ์ที่เกิดข้อผิดพลาดในไฟล์เสียง

ส่วนประกอบของการทดลอง



กล้องเว็ปแคมก็มีไมค์อยู่ในตัว





TP-Link_1939





ขั้นตอนการ Deploy (โหมด Network)

1. สร้าง App ขึ้นมารองรับโดยตั้งชื่อว่า AC_testing
2. ปิด App เข้า Terminal พร้อมที่จะทำการ Access บน Command Line บนลีนุกซ์
3. เตรียมไฟล์ AC_model.eim จากแฟลชไดร์ฟ เสียบเข้า Dongle ที่ต่อเข้ากับ Arduino UNO Q
4. Terminal เข้าตัวบอร์ด Arduino UNO Q โดยโหมด Network Mode
5. ที่หน้ารูธเริ่มด้วยคำสั่งตรวจสอบและ mount ตัว Flash Drive ดังนี้

```
lsblk
sudo blkid /dev/sda1
sudo mkdir -p /media/usb
sudo mount /dev/sda1 /media/usb
df –h
```

ขั้นตอนการ Deploy (โหมด Network)

6. ทำการตรวจสอบว่ามีห้อง ei-models หรือยังโดยใช้คำสั่ง

ls -l /home/arduino/.arduino-bricks/ei-models/

ถ้ายังใช้คำสั่ง

mkdir -p /home/arduino/.arduino-bricks/ei-models/

7. ทำการคัดลอกไฟล์ .eim ที่ได้ทำการเทรนมาบน Edge Impulse

cp /media/usb/AC_model.eim /home/arduino/.arduino-bricks/ei-models/

8. ตรวจสอบว่ามีไฟล์อยู่ในโฟลเดอร์หรือไม่

ls -l /home/arduino/.arduino-bricks/ei-models/

9. เปิดไฟล์ app.yaml ของ App ขึ้นมา

nano /home/arduino/ArduinoApps/ac_testing/app.yaml

ขั้นตอนการ Deploy (โหมด Network)

10. เพิ่มหรือแก้ไขไฟล์ app.yaml ใน App

```
variables: {
  EI_AUDIO_CLASSIFICATION_MODEL: "/home/arduino/.arduino-bricks/ei-models/AC_model.eim"
}
```

11. หลังจากแก้ไขแล้ว: บันทึกไฟล์ (ใน nano: Ctrl+O แล้ว Enter, จากนั้น Ctrl+X เพื่อออก)

12. ถอดยูเอสบีออก

```
sudo umount /media/usb
```

```
Activating python virtual environment
2025-11-27 06:21:15.055 WARNING - [MainThread] arduino.app_internal.core.ei:  [AudioClassification] Host: ei-audio-classifier-runner - Ports: 1337 - URL: http://ei-audio-classifier-runner:1337
2025-11-27 06:21:15.082 INFO - [MainThread] Microphone:  Init Microphone with device=USB_MIC_1, sample_rate=8000, channels=1, format=S16_LE, periodsize=1024
2025-11-27 06:21:15.127 INFO - [MainThread] Microphone:  USB microphones found: ['plughw:CARD=Audio,DEV=0']
=== ดู attribute ทั้งหมดของ object ===
axis_count: 1
frequency: 8000
image_input_height: 0
image_input_width: 0
input_features_count: 8000
interval_ms: 0.125
label_count: 2
labels: ['Background', 'dog_barking']
model_type: classification
name: My_AC
project_id: 839010
thresholds: []
2025-11-27 06:21:15.272 WARNING - [MainThread] Microphone:  [__del__] Microphone __del__: cleaning up resources.
2025-11-27 06:21:15.273 WARNING - [MainThread] Microphone:  Microphone is not recording, nothing to stop.
exited with code 0
```

/home/arduino/.arduino-bricks/ei-models/

AC_model.eim



Dog_Barking Background

1

defualt_model.eim

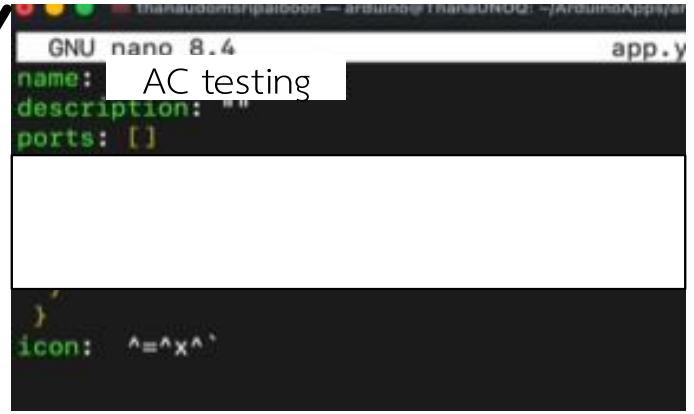
Grass_Breaking

Background



AC testing

app.yaml



/home/arduino/.arduino-bricks/ei-models/

AC_model.eim



Dog_Barking Background

defualt_model.eim

Grass_Breaking

Background



AC testing

app.yaml

